LAPORAN PRAKTIKUM
TEKNOLOGI IOT II

Modul 6 Latihan

Dosen Pengampu:

1. Dr. Ir. Septafiansyah Dwi Putra - NIP : 198809112009121001
2. Agus Ambarwari, S.Pd., M.Kom. - NIP : 199208082019031020



Disusun oleh :

Ananda Haris Rahmadi (25782032)

PROGRAM STUDI TEKNOLOGI REKAYASA INTERNET

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI LAMPUNG

2026

Latihan 1: Analisis Arsitektur & Perbandingan Protokol

1. Efisiensi penggunaan kapasitas bandwidth pada paket data seluler:
    - Jawaban: MQTT menggunakan header protokol yang sangat ringkas dengan ukuran minimum hanya 2 byte serta berbasis arsitektur publish-subscribe, sehingga sangat hemat kuota data seluler. Sebaliknya, HTTP bersifat berbasis teks (text-based) dan stateless dengan header yang panjang pada setiap pertukaran data (request/response), yang menyebabkan pemborosan bandwidth yang signifikan pada jaringan seluler.
2. Beban komputasi mikrokontroler (CPU overhead) saat menangani pertukaran data dua arah:
    - Jawaban: MQTT memelihara koneksi TCP jangka panjang yang persisten, sehingga mikrokontroler hanya perlu memproses sedikit paket data kontrol tanpa harus melakukan handshake TCP berulang-ulang, yang membuat CPU overhead-nya sangat rendah. Sementara itu, HTTP mewajibkan perangkat membuka koneksi TCP baru dan memparsing teks header yang masif setiap kali mengambil atau mengirim data, sehingga sangat membebani memori dan siklus clock CPU mikrokontroler.
3. Kemampuan pengiriman data secara simultan ke banyak perangkat penerima (1-to-N distribution):
    - Jawaban: MQTT menggunakan arsitektur Publish-Subscribe berbasis Broker, di mana satu Publisher cukup mengirim data satu kali ke broker dan broker akan mendistribusikannya secara simultan ke banyak Subscriber melalui pencocokan topik (topic matching). Di sisi lain, HTTP bersifat point-to-point (1-to-1), sehingga server harus melayani permintaan terpisah (polling) dari masing-masing klien secara mandiri yang tidak efisien untuk distribusi massal.

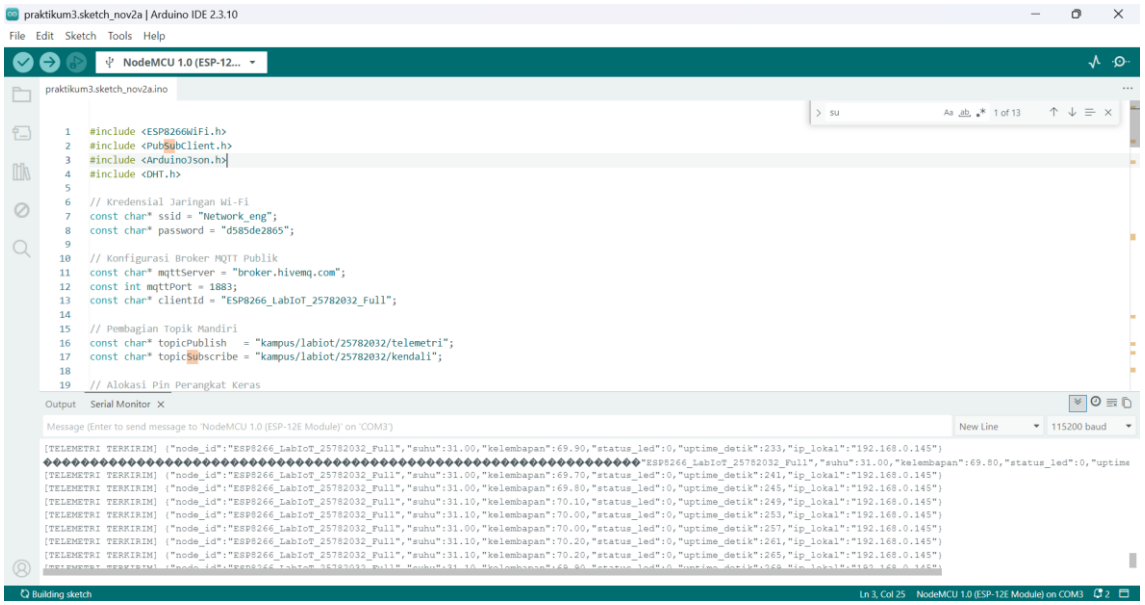
Latihan 2: Analisis Hierarki Topik & Karakter Wildcard

Berdasarkan struktur topik dasar pabrik/<line>/<mesin>/<sensor>:

1. Memantau parameter suhu dari seluruh mesin di semua lini produksi:
    - Jawaban: pabrik/+/+/suhu
2. Memantau seluruh parameter sensor (suhu, getaran, status) khusus untuk mesin m1 di lini line2:
    - Jawaban: pabrik/line2/m1/#

3. Memantau seluruh lalu lintas data tanpa terkecuali dari seluruh lini dan seluruh mesin di pabrik tersebut:
    - Jawaban: pabrik/#

Latihan 3: Memantau Data Masuk (Telemetry Monitoring)



Kode:

```
#include <ESP8266WiFi.h>
#include <PubSubClient.h>
#include <ArduinoJson.h>
#include <DHT.h>

// Kredensial Jaringan Wi-Fi
const char* ssid = "Network_eng";
const char* password = "d585de2865";

// Konfigurasi Broker MQTT Publik
const char* mqttServer = "broker.hivemq.com";
const int mqttPort = 1883;
const char* clientId = "ESP8266_LabIoT_25782032_Full";

// Pembagian Topik Mandiri
const char* topicPublish   = "kampus/labiot/25782032/telemetri";
const char* topicSubscribe = "kampus/labiot/25782032/kendali";
```

```
// Alokasi Pin Perangkat Keras
const byte dhtPin = 2; // Pin D4 (GPIO 2)
const byte ledPin = 5; // Pin D1 (GPIO 5)
#define DHTTYPE DHT22

DHT dht(dhtPin, DHTTYPE);
WiFiClient espClient;
PubSubClient client(espClient);

unsigned long lastTelemetryTime = 0;
const unsigned long telemetryInterval = 4000; // Siklus kirim 4 detik

void callback(char* topic, byte* payload, unsigned int length) {
  char incomingBuffer[length + 1];
  memcpy(incomingBuffer, payload, length);
  incomingBuffer[length] = '\0';

  Serial.println("----------------------------------------");
  Serial.printf("[KONTROL DITERIMA] Topik: %s | Payload: %s\n", topic,
incomingBuffer);

  StaticJsonDocument<256> doc;
  DeserializationError err = deserializeJson(doc, incomingBuffer);
  if (err) {
    Serial.println("Gagal membedah payload JSON!");
    return;
  }

  if (doc.containsKey("led")) {
    int val = doc["led"];
    digitalWrite(ledPin, val == 1 ? HIGH : LOW);
    Serial.printf("--> [INDIKATOR] Status LED diubah: %s\n", val == 1 ?
"MENYALA" : "PADAM");
  }
  Serial.println("----------------------------------------");
}

void reconnect() {
  while (!client.connected()) {
    Serial.print("Menghubungkan kembali ke Broker...");
    if (client.connect(clientId)) {
      Serial.println(" Sukses Terhubung!");
      client.subscribe(topicSubscribe);
      Serial.printf("Subskripsi Topik: %s\n", topicSubscribe);
    } else {
      Serial.printf("Gagal (rc=%d). Coba lagi dalam 4 detik...\n",
client.state());
```

```
      delay(4000);
    }
  }
}

void setup() {
  Serial.begin(115200);
  pinMode(ledPin, OUTPUT);
  digitalWrite(ledPin, LOW);

  dht.begin();

  WiFi.mode(WIFI_STA);
  WiFi.begin(ssid, password);
  Serial.print("Menghubungkan Wi-Fi");
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("\nWi-Fi Terkoneksi!");
  Serial.print("IP Address: ");
  Serial.println(WiFi.localIP());

  client.setServer(mqttServer, mqttPort);
  client.setCallback(callback);
}

void loop() {
  if (!client.connected()) {
    reconnect();
  }
  client.loop(); // Memelihara sesi TCP dan memproses callback masuk

  unsigned long now = millis();
  if (now - lastTelemetryTime >= telemetryInterval) {
    lastTelemetryTime = now;

    float t = dht.readTemperature();
    float h = dht.readHumidity();

    if (!isnan(t) && !isnan(h)) {
      StaticJsonDocument<256> docOut;
      docOut["node_id"] = clientId;
      docOut["suhu"] = serialized(String(t, 2));
      docOut["kelembapan"] = serialized(String(h, 2));
      docOut["status_led"] = digitalRead(ledPin);
```

```
      // Penambahan parameter dinamis sesuai instruksi Latihan 3
      docOut["uptime_detik"] = millis() / 1000;
      docOut["ip_lokal"] = WiFi.localIP().toString();

      char buffer[256];
      serializeJson(docOut, buffer);

      bool success = client.publish(topicPublish, buffer);
      if (success) {
        Serial.printf("[TELEMETRI TERKIRIM] %s\n", buffer);
      } else {
        Serial.println("[ERROR] Gagal memublikasikan telemetri!");
      }
    }
  }
}
```

Latihan 4: Analisis Mekanisme Reconnect & Loop Blocking

1. Apa konsekuensi teknis yang terjadi terhadap responsivitas penerimaan perintah callback jika seorang pengembang menyisipkan instruksi delay(6000) di dalam fungsi loop()?

    - Jawaban: Mikrokontroler akan berhenti total (*blocking*) dari mengeksekusi instruksi apa pun selama 6 detik tersebut. Akibatnya, mikrokontroler menjadi tidak responsif terhadap perintah masuk dari broker MQTT, koneksi berisiko mengalami *timeout* atau terputus karena gagal mengirim paket pemeliharaan (*pingreq*), dan siklus pengiriman telemetri menjadi terganggu.

2. Bagaimana fungsi reconnect() bekerja memulihkan koneksi saat router Wi-Fi atau broker MQTT sempat terputus sesaat?

    - **Jawaban:** Fungsi reconnect() memantau status koneksi menggunakan client.connected(). Jika bernilai false, program masuk ke blok rekoneksi dan mencoba melakukan *handshake* ulang ke broker via client.connect(clientId). Apabila berhasil terhubung kembali, fungsi ini secara otomatis mendaftarkan ulang subskripsi topik perintah melalui client.subscribe(topicSubscribe) agar langganan data kendali aktif kembali. Jika masih gagal, sistem akan menunggu selama 4 detik sebelum mencoba menghubungkan ulang secara berkala.